08. Narwhal 통합 및 설정

목차

1. Narwhal 통합 아키텍처
2. 컨센서스 선택기
3. 설정 확장
4. 노드 통합
5. 마이그레이션 가이드
6. 호환성 보장

Narwhal 통합 아키텍처

통합 개요

Autobahn BFT는 기존 Narwhal 인프라와 완전히 호환되도록 설계되었습니다. 기존 Narwhal 배포에서 설정 변경만으로 Autobahn을 활성화할 수 있습니다.

```
┌─────────────────────────────────────────────────────────────┐
│ Node Configuration │
├─────────────────────────────────────────────────────────────┤
│ consensus_algorithm: "Autobahn" | "Narwhal" │
│ autobahn: { ... } // Autobahn 전용 설정 │
└─────────────────────────────────────────────────────────────┘
```

```
│
▼
┌──────────────────────────────────────────────────────────────┐
│ ConsensusSelector │
├──────────────────────────────────────────────────────────────┤
│ ┌──────────────────┐ ┌──────────────────────┐ │
│ │ NarwhalFactory │ │ AutobahnFactory │ │
│ │ (Bullshark) │ │ (AutobahnController)│ │
│ └──────────────────┘ └──────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
│
▼
┌──────────────────────────────────────────────────────────────┐
│ ConsensusWrapper │
│ - process_certificate() │
│ - update_committee() │
└──────────────────────────────────────────────────────────────┘
```

컨센서스 선택기

ConsensusSelector 구현

```
// node/src/consensus_selector.rs
```

```rust
use config::{AutobahnParameters, Committee, ConsensusAlgorithm, Parameters};
use consensus::{
bullshark::Bullshark,
metrics::ConsensusMetrics,
ConsensusProtocol, ConsensusState, ConsensusOutput,
};
use std::sync::Arc;
use types::{ConsensusStore, Certificate, SequenceNumber, StoreResult};

/// 설정에 따른 컨센서스 프로토콜 생성을 위한 트레이트
pub trait ConsensusFactory {
type Protocol: ConsensusProtocol + Send + 'static;
fn create_consensus(
committee: Committee,
store: Arc<ConsensusStore>,
parameters: &Parameters,
metrics: Arc<ConsensusMetrics>,
) -> Self::Protocol;
}

/// Narwhal 컨센서스 (Bullshark) 팩토리
pub struct NarwhalFactory;
```

```rust
impl ConsensusFactory for NarwhalFactory {
type Protocol = Bullshark;
fn create_consensus(
committee: Committee,
store: Arc<ConsensusStore>,
parameters: &Parameters,
_metrics: Arc<ConsensusMetrics>,
) -> Self::Protocol {
Bullshark::new(committee, store, parameters.gc_depth)
}
}

/// Autobahn 컨센서스 팩토리
pub struct AutobahnFactory;

impl ConsensusFactory for AutobahnFactory {
type Protocol = autobahn::AutobahnController;
fn create_consensus(
committee: Committee,
store: Arc<ConsensusStore>,
parameters: &Parameters,
metrics: Arc<ConsensusMetrics>,
```

```
) -> Self::Protocol {
let autobahn_params = parameters.autobahn.as_ref()
.expect("Autobahn parameters must be provided when using Autobahn consensus");
let config = autobahn::AutobahnConfig::from_parameters(autobahn_params);
autobahn::AutobahnController::new(
committee,
store,
config,
metrics,
parameters.gc_depth,
)
}
}
```

ConsensusWrapper 구현

```
/// 다양한 컨센서스 프로토콜을 위한 래퍼 열거형
pub enum ConsensusWrapper {
Narwhal(Bullshark),
Autobahn(autobahn::AutobahnController),
}

impl ConsensusProtocol for ConsensusWrapper {
```

```
fn process_certificate(
&mut self,
state: &mut ConsensusState,
consensus_index: SequenceNumber,
certificate: Certificate,
) -> StoreResult<Vec<ConsensusOutput>> {
match self {
ConsensusWrapper::Narwhal(bullshark) => {
bullshark.process_certificate(state, consensus_index, certificate)
}
ConsensusWrapper::Autobahn(autobahn) => {
autobahn.process_certificate(state, consensus_index, certificate)
}
}
}

fn update_committee(&mut self, new_committee: Committee) -> StoreResult<()> {
match self {
ConsensusWrapper::Narwhal(bullshark) => {
bullshark.update_committee(new_committee)
}
ConsensusWrapper::Autobahn(autobahn) => {
autobahn.update_committee(new_committee)
```

```
}
}
}
}

/// 설정에 따른 적절한 컨센서스 프로토콜 생성
pub fn create_consensus_protocol(
committee: Committee,
store: Arc<ConsensusStore>,
parameters: &Parameters,
metrics: Arc<ConsensusMetrics>,
) -> ConsensusWrapper {
match parameters.consensus_algorithm {
ConsensusAlgorithm::Narwhal => {
ConsensusWrapper::Narwhal(NarwhalFactory::create_consensus(committee, store, par
}
ConsensusAlgorithm::Autobahn => {
ConsensusWrapper::Autobahn(AutobahnFactory::create_consensus(committee, store, p
}
}
}
```

설정 확장

Parameters 구조 확장

```rust
// config/src/lib.rs


/// Narwhal 노드 속성을 보유하는 구조체
#[derive(Clone, Debug, Deserialize, Serialize)]
pub struct Parameters {
// 기존 Narwhal 매개변수들...
pub header_size: usize,
pub max_header_delay: Duration,
pub gc_depth: u64,
// ... 기타 기존 필드들


/// 사용할 컨센서스 알고리즘 (Narwhal 또는 Autobahn)
pub consensus_algorithm: ConsensusAlgorithm,
/// Autobahn 전용 매개변수 (consensus_algorithm이 Autobahn일 때만 사용)
#[serde(skip_serializing_if = "Option::is_none")]
pub autobahn: Option<AutobahnParameters>,
}


/// 컨센서스 알고리즘 선택
```

```rust
#[derive(Clone, Debug, Deserialize, Serialize, PartialEq, Eq)]
pub enum ConsensusAlgorithm {
/// 원래 Narwhal 컨센서스 (Bullshark) 사용
Narwhal,
/// Autobahn BFT 컨센서스 사용
Autobahn,
}

impl Default for ConsensusAlgorithm {
fn default() -> Self {
Self::Narwhal
}
}

/// Autobahn 전용 설정 매개변수
#[derive(Clone, Debug, Deserialize, Serialize)]
pub struct AutobahnParameters {
/// Fast Path 컨센서스를 위한 기본 타임아웃
#[serde(with = "duration_format")]
pub fast_path_timeout: Duration,
/// Fast Path에 허용되는 최대 타임아웃
#[serde(with = "duration_format")]
pub max_fast_path_timeout: Duration,
```

```rust
/// Fast Path에 허용되는 최소 타임아웃
#[serde(with = "duration_format")]
pub min_fast_path_timeout: Duration,
/// 타임아웃 적응 계수 (0.0에서 1.0)
pub timeout_adaptation_factor: f64,
/// 유지할 타임아웃 히스토리 항목의 최대 수
pub timeout_history_size: usize,
/// 집계를 위한 메시지 배치 크기 임계값
pub message_batch_size: usize,
/// 메시지 배치 타임아웃
#[serde(with = "duration_format")]
pub message_batch_timeout: Duration,
/// 비잔틴 행동 감지 활성화
pub enable_byzantine_detection: bool,
/// Slow Path로 전환하기 위한 임계값 (감지된 비잔틴 행동 수)
pub byzantine_threshold: usize,
}

impl Default for AutobahnParameters {
fn default() -> Self {
Self {
fast_path_timeout: Duration::from_millis(100),
max_fast_path_timeout: Duration::from_secs(5),
```

```
min_fast_path_timeout: Duration::from_millis(50),

timeout_adaptation_factor: 0.1,

timeout_history_size: 100,

message_batch_size: 100,

message_batch_timeout: Duration::from_millis(50),

enable_byzantine_detection: true,

byzantine_threshold: 1,

}

}

}
```

설정 유효성 검사

```
impl Parameters {

pub  fn  validate_autobahn_config(&self) -> Result<(), String> {

if  self.consensus_algorithm == ConsensusAlgorithm::Autobahn {

if  let Some(autobahn_params) = &self.autobahn {

// 타임아웃 범위 검증

if  autobahn_params.min_fast_path_timeout >= autobahn_params.max_fast_path_timeo

return Err("min_fast_path_timeout must be less than max_fast_path_timeout".to_st

}

if  autobahn_params.fast_path_timeout < autobahn_params.min_fast_path_timeout

|| autobahn_params.fast_path_timeout > autobahn_params.max_fast_path_timeout {
```

```
return Err("fast_path_timeout must be between min and max timeouts".to_string())
}
// 적응 계수 검증
if  autobahn_params.timeout_adaptation_factor < 0.0 || autobahn_params.timeout_a
return Err("timeout_adaptation_factor must be between 0.0 and 1.0".to_string());
}
Ok(())
} else {
Err("Autobahn parameters must be provided when using Autobahn consensus".to_stri
}
} else {
Ok(())
}
}
}
```

노드 통합

Node::spawn_consensus 업데이트

```
// node/src/lib.rs

impl Node {
```

```
async fn spawn_consensus<State>(
name: PublicKey,
network: oneshot::Receiver<P2pNetwork>,
worker_cache: SharedWorkerCache,
committee: SharedCommittee,
store: &NodeStorage,
parameters: Parameters,
execution_state: State,
tx_reconfigure: &watch::Sender<ReconfigureNotification>,
rx_new_certificates: metered_channel::Receiver<Certificate>,
tx_feedback: metered_channel::Sender<Certificate>,
registry: &Registry,
) -> SubscriberResult<Vec<JoinHandle<()>>>
where
State: ExecutionState + Send + Sync + 'static,
{
let consensus_metrics = Arc::new(ConsensusMetrics::new(registry));
let channel_metrics = ChannelMetrics::new(registry);


let (tx_sequence, rx_sequence) =
metered_channel::channel(Self::CHANNEL_CAPACITY, &channel_metrics.tx_sequence);


// 복원된 컨센서스 출력 확인
```

```
let  restored_consensus_output = get_restored_consensus_output(
store.consensus_store.clone(),
store.certificate_store.clone(),
&execution_state,
)
.await?
.into_iter()
.sorted_by(|a, b| a.consensus_index.cmp(&b.consensus_index))
.collect::<Vec<ConsensusOutput>>();


// 설정에 따른 컨센서스 프로토콜 생성
let  ordering_engine = crate::consensus_selector::create_consensus_protocol(
(**committee.load()).clone(),
store.consensus_store.clone(),
&parameters,
consensus_metrics.clone(),
);


// 컨센서스 스폰
let  consensus_handles = Consensus::spawn(
(**committee.load()).clone(),
store.consensus_store.clone(),
store.certificate_store.clone(),
```

```
tx_reconfigure.subscribe(),
rx_new_certificates,
tx_feedback,
tx_sequence,
ordering_engine,
consensus_metrics.clone(),
parameters.gc_depth,
);


// 실행자 스폰
let  executor_handles = Executor::spawn(
name,
network,
worker_cache,
(**committee.load()).clone(),
execution_state,
tx_reconfigure,
rx_sequence,
registry,
restored_consensus_output,
)?;


Ok(executor_handles
```

```
.into_iter()
.chain(std::iter::once(consensus_handles))
.collect())
}
}
```

마이그레이션 가이드

단계별 마이그레이션

1단계: 설정 업데이트

```
{
"consensus_algorithm": "Autobahn",
"autobahn": {
"fast_path_timeout": "150ms",
"max_fast_path_timeout": "10s",
"min_fast_path_timeout": "50ms",
"timeout_adaptation_factor": 0.1,
"timeout_history_size": 100,
"message_batch_size": 100,
"message_batch_timeout": "75ms",
"enable_byzantine_detection": true,
"byzantine_threshold": 1
```

```
}

}
```

2단계: 점진적 롤아웃

```
impl MigrationManager {

pub  async  fn  gradual_rollout(&self, nodes: Vec<NodeId>) -> Result<(), Migrati

// 1. 단일 노드에서 테스트

let  test_node = &nodes[0];

self.enable_autobahn_on_node(test_node).await?;

self.monitor_performance(test_node, Duration::from_hours(1)).await?;

// 2. 소규모 그룹으로 확장

let  small_group = &nodes[0..nodes.len()/4];

for  node  in  small_group {

self.enable_autobahn_on_node(node).await?;

}

self.monitor_performance_group(small_group, Duration::from_hours(2)).await?;

// 3. 전체 배포

for  node  in &nodes[nodes.len()/4..] {

self.enable_autobahn_on_node(node).await?;

tokio::time::sleep(Duration::from_secs(30)).await; // 점진적 배포

}

Ok(())

}
```

```
}
```

3단계: 성능 모니터링

```
impl MigrationManager {
async fn monitor_performance(&self, node: &NodeId, duration: Duration) -> Resu
let start = Instant::now();
while start.elapsed() < duration {
let metrics = self.get_node_metrics(node).await?;
// 성능 저하 확인
if metrics.consensus_latency > Duration::from_millis(500) {
return Err(MigrationError::PerformanceDegradation);
}
// 오류율 확인
if metrics.error_rate > 0.05 {
return Err(MigrationError::HighErrorRate);
}
tokio::time::sleep(Duration::from_secs(10)).await;
}
Ok(())
}
}
```

롤백 절차

```
impl MigrationManager {
pub  async  fn  rollback_to_narwhal(&self, nodes: Vec<NodeId>) -> Result<(), Mig
for  node  in  nodes {
// 설정을 Narwhal로 되돌리기
let  config = NodeConfig {
consensus_algorithm: ConsensusAlgorithm::Narwhal,
autobahn: None,
..self.get_current_config(&node).await?
};
self.update_node_config(&node, config).await?;
self.restart_node(&node).await?;
// 롤백 확인
self.verify_narwhal_operation(&node).await?;
}
Ok(())
}
}
```

호환성 보장

인증서 형식 호환성

```
impl AutobahnCertificate {
/// Narwhal Certificate에서 AutobahnCertificate 생성
pub fn from_narwhal_certificate(cert: Certificate) -> Self {
Self {
base_certificate: cert,
fast_path_info: None,
timeout_info: None,
autobahn_metadata: None,
}
}
/// 기본 Narwhal Certificate 추출
pub fn to_narwhal_certificate(&self) -> Certificate {
self.base_certificate.clone()
}
/// Narwhal과의 호환성을 위한 기본 Certificate 반환
pub fn base_certificate(&self) -> &Certificate {
&self.base_certificate
}
}
```

저장소 호환성

```
impl AutobahnController {
```

```
/// 기존 Narwhal 저장소와 호환되는 생성자
pub fn from_narwhal_store(
committee: Committee,
narwhal_store: Arc<ConsensusStore>,
config: AutobahnConfig,
metrics: Arc<ConsensusMetrics>,
gc_depth: Round,
) -> Self {
// Narwhal 저장소를 그대로 사용
Self::new(committee, narwhal_store, config, metrics, gc_depth)
}
}
```

네트워크 프로토콜 호환성

```
impl AutobahnController {
/// Narwhal 네트워크 프로토콜과 호환되는 메시지 처리
pub async fn handle_narwhal_message(&mut self, message: NarwhalMessage) -> R
match message {
NarwhalMessage::Certificate(cert) => {
// Narwhal 인증서를 Autobahn 인증서로 변환
let autobahn_cert = AutobahnCertificate::from_narwhal_certificate(cert);
self.process_certificate(autobahn_cert).await?;
```

```
}
NarwhalMessage::Vote(vote) => {
// Narwhal 투표를 Fast Path 투표로 변환
let fast_path_vote = FastPathVote::from_narwhal_vote(vote);
self.process_vote(fast_path_vote).await?;
}
_ => {
// 기타 Narwhal 메시지는 Slow Path에서 처리
let mut slow_path = self.slow_path.write().await;
slow_path.handle_narwhal_message(message).await?;
}
}
Ok(())
}
}
```

API 호환성

```
impl AutobahnController {
/// Narwhal ConsensusProtocol 트레이트와 완전 호환
pub fn as_consensus_protocol(&mut self) -> &mut dyn ConsensusProtocol {
self
```

```
}
}

// 기존 Narwhal 코드와의 호환성을 위한 타입 별칭
pub type NarwhalCompatibleConsensus = ConsensusWrapper;
```

설정 마이그레이션 유틸리티

```
pub struct ConfigMigrator;

impl ConfigMigrator {
/// 기존 Narwhal 설정을 Autobahn 설정으로 변환
pub fn migrate_narwhal_to_autobahn(narwhal_params: Parameters) -> Parameters {
let autobahn_params = AutobahnParameters {
fast_path_timeout: narwhal_params.max_header_delay,
max_fast_path_timeout: narwhal_params.max_header_delay * 5,
min_fast_path_timeout: narwhal_params.max_header_delay / 2,
timeout_adaptation_factor: 0.1,
timeout_history_size: 100,
message_batch_size: 100,
message_batch_timeout: narwhal_params.max_batch_delay,
enable_byzantine_detection: true,
byzantine_threshold: 1,
```

```
};

Parameters {

consensus_algorithm: ConsensusAlgorithm::Autobahn,

autobahn: Some(autobahn_params),

..narwhal_params

}

}

/// Autobahn 설정을 Narwhal 설정으로 되돌리기

pub fn rollback_autobahn_to_narwhal(autobahn_params: Parameters) -> Parameters

Parameters {

consensus_algorithm: ConsensusAlgorithm::Narwhal,

autobahn: None,

..autobahn_params

}

}

}
```

Narwhal 통합은 기존 인프라와의 완전한 호환성을 보장하면서 Autobahn BFT의 고급 기능을 제공합니다. 점진적 마이그레이션과 롤백 메커니즘을 통해 안전한 전환이 가능합니다.